

DOCUMENTATION

Automated Matching of Customer Challenges to Intershop Customer References

Onur Yazicioglu
Prototype · April 2026

1. Goal of the Solution

The goal was to develop a system that supports sales representatives and customers in finding relevant Intershop customer references for specific challenges. Instead of searching manually, the user enters a challenge — for example, "We want to sell in 20 countries" — and the system automatically returns the most relevant references.

In addition, the system offers further features to support the sales process: users can view all previously stored challenges, retrieve a visual overview showing how many references each challenge covers, and automatically receive a product recommendation for a specific challenge. This recommendation is generated by a Large Language Model that matches the challenge against the available Intershop products and returns the most suitable options with justification.

2. How the Solution Works

2.1 Data Collection – Web Scraper

Since the Intershop website does not offer a structured data interface, two scrapers were developed, each automatically extracting different types of content and saving them locally.

The first scraper reads the Intershop references page and saves the data for each company in a separate JSON file. Each file contains a short description of the customer benefit as well as metadata such as company name, logo URL, and, where available, a link to the detail page or the customer's website. This format is directly compatible with modern AI frameworks and can be loaded into a vector database without conversion.

The second scraper traverses several Intershop solution pages — including B2B, B2C, B2X, Manufacturing, Wholesale, and Retail — and extracts headings, descriptions, and feature lists. The collected content is saved in a single text file (products.txt), structured by source URL. This file later serves as the knowledge base for product recommendations: it is passed along with the user's challenge to the language model so that it can generate a well-founded, product-specific recommendation.

2.2 Vector Database – Semantic Search

The stored reference texts are converted into numerical vectors (embeddings) and saved in a local vector database. When searching, the user's query is encoded in the same way and compared with the stored references — not through keyword matching, but through semantic

similarity. This means: "We have a very large number of products" will also find references that mention a "large product catalog" or "80,000 items", even if those exact words do not appear in the search query.

2.3 Challenges – Deduplication and Tagging

New challenges are checked for similarity against existing ones before being saved. If the semantic similarity exceeds a defined threshold, the new challenge is rejected and the user is referred to the existing one. This keeps the list of challenges clean and free of duplicates.

If a challenge is accepted, it is automatically matched against all references that are relevant in content. These references are tagged accordingly, so that later search queries only return results that have been explicitly verified as relevant to the given challenge.

2.4 Filtered Search Results

When a search query is made, up to 15 candidates are first retrieved from the vector database. Each candidate is then filtered based on stored metadata: only references that were classified as relevant during the tagging step make it into the results. Of these, a maximum of 5 are returned to the user, sorted by semantic similarity.

2.5 Product Recommendation

Via a separate endpoint, the user can request a product recommendation for a known challenge. The system sends the challenge together with the available Intershop product descriptions to a Large Language Model (Azure AI Foundry). The model analyzes the context and returns a reasoned recommendation of which Intershop products best match the customer's situation.

2.6 Visualization

A visualization endpoint displays in the terminal how many references are assigned to each stored challenge. This provides a quick overview of which topics are particularly well covered in the reference portfolio.

3. System Architecture

The system is modular in design and consists of the following components:

- **Scrapers:** Extraction of customer and product data from the Intershop website
- **Embedding Module:** Central component for text vectorization, loaded once and reused
- **Challenge Module:** Management, deduplication, and tagging of challenges
- **RAG Module:** Construction and querying of the vector database including metadata filtering
- **API Server:** FastAPI application with 6 endpoints
- **CLI Client:** Command-line tool for all endpoints

4. Technologies Used

AI Assistants (Vibe Coding)

AI-assisted coding tools were used for this project, both for initial brainstorming and for implementation and documentation. Specifically, Claude Sonnet 4.6 and GPT-5.3 Instant were used.

Python

Project language. Chosen for its broad support in the AI and data space, and for the availability of all required libraries as stable, well-documented packages.

FastAPI

Framework for the REST API server. FastAPI enables rapid development of robust APIs with automatic validation and interactive documentation. It is asynchronous-capable and well suited for AI-powered backends.

sentence-transformers / all-MiniLM-L6-v2

Embedding model for converting texts into semantic vectors. The model runs entirely locally without an API key, is fast, and is optimized for semantic similarity search. The same model is used for deduplication, tagging, and vector search, keeping the vector space consistent. A small, local neural network model was deliberately chosen to demonstrate the ability to integrate AI models directly and use them productively without dependency on external APIs.

ChromaDB

Local vector database for storing and querying reference embeddings. ChromaDB persists data to disk and offers direct integration with LangChain. No separate database installation is required.

LangChain

Framework for orchestrating the RAG pipeline. LangChain provides the Document abstraction through which JSON files with content and metadata are structured and passed to ChromaDB.

BeautifulSoup / Requests

Libraries for web scraping. Requests handles the HTTP retrieval of the Intershop page; BeautifulSoup handles structured parsing of the HTML. The combination is lightweight and well suited for server-side rendered pages.

scikit-learn

Used for cosine similarity calculation during challenge deduplication. The library is numerically stable and widely used.

Azure AI Foundry (Azure OpenAI)

LLM service for product recommendations. The challenge and product descriptions are passed to a GPT model, which returns a contextualized recommendation. Azure AI Foundry was chosen because it is already licensed and GDPR-compliant in many enterprise environments.

A cloud-based AI platform was deliberately chosen to demonstrate the ability to professionally use cloud-based AI and machine learning services.

5. API Endpoints Overview

POST /challenge/new: Submit a new challenge (incl. deduplication, tagging, automatic search)

GET /search: Reference search for a known challenge

GET /visualization: Overview: challenge → number of tagged references

GET /product-recommendation: Product recommendation via Azure LLM

GET /challenges: List of all stored challenges

GET /endpoints: Overview of all available endpoints

6. Extensibility

The system is intentionally kept modular. Individual components can be replaced or extended independently:

- **Frontend:** Currently operated via a CLI tool. Since the system is REST API-based, a web frontend can be added at any time.
- **Embedding Model:** Can be replaced by advanced OpenAI or Azure embeddings by changing a single line.
- **Reference Detail Pages:** The "Read more" links stored in the JSON files have not yet been used. The linked detail pages could additionally be scraped and included in the search. Since these texts are much longer and structurally different from the short reference descriptions, it would make sense to build two separate vector databases.
- **Zero-Shot Classification:** Instead of using embeddings for assigning challenges to references, zero-shot classifiers could be used. This would allow more precise categorization, but at the cost of flexibility — adding new challenges would become more complex.
- **Unit Tests:** Adding unit tests for the individual modules — especially for the deduplication logic, tagging, and the RAG pipeline — would increase the reliability of the system and safeguard future extensions.